

WEEK -2
SORTING TECHNIQUES

1) Bubble Sort:

Program:

```
#include<stdio.h>
#define size 5
void bubblesort(int arr[], int n){
for(int i=0;i<n;i++){
for(int j=0;j<n-i-1;j++){
if(arr[j]>arr[j+1]){
int temp=arr[j];
arr[j]=arr[j+1];
arr[j+1]=temp; }
}
}
}
int main(){
int arr[size];
for(int i=0;i<size;i++){
printf("enter element %d: ",i);
scanf("%d",&arr[i]);
}
bubblesort(arr,size);
for(int i=0;i<size;i++){
printf("%d\\n",arr[i]); }
return 0;
}
```

Output:

```
amma@amma11:~$ gcc bubble_sort.c -o bubble_sort
amma@amma11:~$ ./bubble_sort
enter element 0: 6
enter element 1: 9
enter element 2: 8
enter element 3: 1
enter element 4: 3
1
3
6
8
9
```

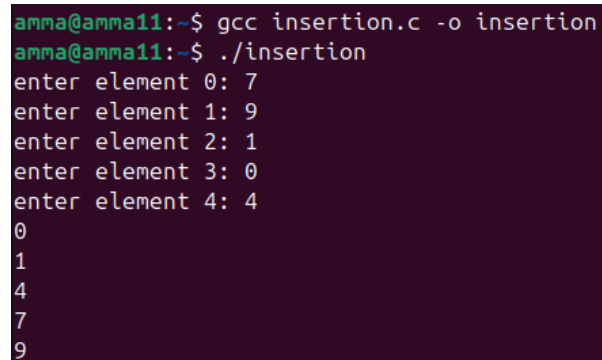
2) Insertion Sort:

Program:

```
#include<stdio.h>
#define size 5
void insertionsort(int arr[], int n){
for(int i=1;i<n;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
}

int main(){
    int arr[size];
    for(int i=0;i<size;i++){
        printf("enter element %d: ",i);
        scanf("%d",&arr[i]);
    }
    insertionsort(arr,size);
    for(int i=0;i<size;i++){
        printf("%d\\n",arr[i]);
    }
    return 0;
}
```

Output:



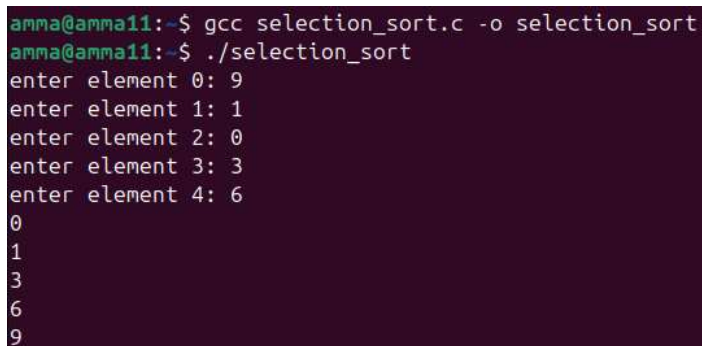
```
amma@amma11:~$ gcc insertion.c -o insertion
amma@amma11:~$ ./insertion
enter element 0: 7
enter element 1: 9
enter element 2: 1
enter element 3: 0
enter element 4: 4
0
1
4
7
9
```

3) Selection Sort:

Program:

```
#include<stdio.h>
#define size 5
void selectionsort(int arr[], int n){
for(int i=0;i<n-1;i++){
    int min=i;
    for(int j=i+1;j<n;j++){
        if(arr[j]<arr[min]){
            min=j;
        }
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
}
int main(){
    int arr[size];
    for(int i=0;i<size;i++){
        printf("enter element %d: ",i);
        scanf("%d",&arr[i]);
    }
    selectionsort(arr,size);
    for(int i=0;i<size;i++){
        printf("%d\n",arr[i]);
    }
    return 0;
}
```

Output:



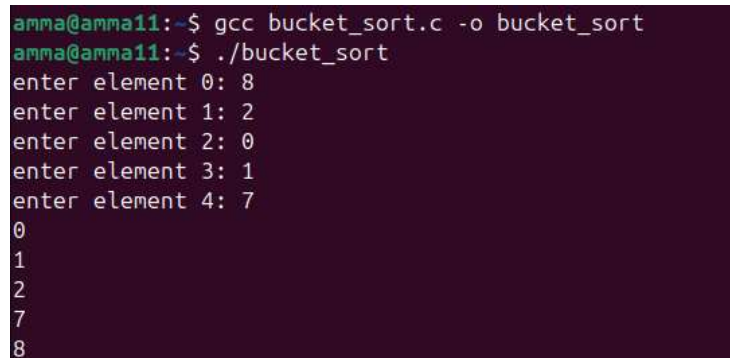
```
amma@amma11:~$ gcc selection_sort.c -o selection_sort
amma@amma11:~$ ./selection_sort
enter element 0: 9
enter element 1: 1
enter element 2: 0
enter element 3: 3
enter element 4: 6
0
1
3
6
9
```

4) Bucket Sort:

Program:

```
#include<stdio.h>
#define size 5
void bucketsort(int arr[], int n){
    int bucket[10]={0};
    for(int i=0;i<n;i++){
        bucket[arr[i]]++;
    }
    int index=0;
    for(int i=0;i<10;i++){
        while(bucket[i]--){
            arr[index++]=i;
        }
    }
}
int main(){
    int arr[size];
    for(int i=0;i<size;i++){
        printf("enter element %d: ",i);
        scanf("%d",&arr[i]);
    }
    bucketsort(arr,size);
    for(int i=0;i<size;i++){
        printf("%d\\n",arr[i]);
    }
    return 0;
}
```

Output:



```
amma@amma11:~$ gcc bucket_sort.c -o bucket_sort
amma@amma11:~$ ./bucket_sort
enter element 0: 8
enter element 1: 2
enter element 2: 0
enter element 3: 1
enter element 4: 7
0
1
2
7
8
```

5) Heap Sort:

Program:

```
#include<stdio.h>
#define size 5
void heapify(int arr[], int n, int i){
    int largest=i;
    int left=2*i+1;
    int right=2*i+2;
    if(left<n && arr[left] > arr[largest]){
        largest = left;
    }
    if(right<n && arr[right] > arr[largest]){
        largest = right;
    }
    if(largest!=i){
        int temp=arr[i];
        arr[i] = arr[largest];
        arr[largest]=temp;
        heapify(arr,n,largest);
    }
}

void heapsort(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main() {
    int arr[size];
    for (int i = 0; i < size; i++) {
        printf("enter element %d: ", i);
```

```
        scanf("%d", &arr[i]);
    }
    heapsort(arr, size);
    for (int i = 0; i < size; i++) {
        printf("%d\n", arr[i]);
    }
    return 0;
}
```

Output:

```
amma@amma11:~$ gcc heap_sort.c -o heap_sort
amma@amma11:~$ ./heap_sort
enter element 0: 9
enter element 1: 1
enter element 2: 8
enter element 3: 3
enter element 4: 6
1
3
6
8
9
```